Multimodal Manufacturing Creator

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	SRASHTI SEN	EN22CS301983
02	SOUMYA SHRIVASTAVA	EN22CS301978
03	TANISHKA RAGHUWANSHI	EN22CS3011019
04	TARUN PATIDAR	EN22CS3011034
05	VIDHI PRAJAPATI	EN22CS3011076

Group Name: Group 12D9

Project Number: Agentic-AI_12

Industry Mentor Name: Mr. Ashvin

University Mentor Name: Prof. Pradeep Baniya

Academic Year:2025-2026

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

With the advancement of **Agentic Artificial Intelligence**, systems are no longer limited to a single monolithic AI model. Instead, intelligent workflows can be created using multiple specialized agents that collaborate to solve complex real-world problems.

This project implements a **Multi-Agent Manufacturing Decision System**, where multiple AI agents work together to perform supplier analysis and generate structured reports.

Unlike traditional systems, this architecture divides responsibilities among agents:

- A **Researcher Agent** gathers structured supplier data
- A **Writer Agent** analyzes and converts data into insights

The system demonstrates how **multi-agent collaboration, task orchestration, and structured communication (JSON)** can be used to solve manufacturing decision-making problems efficiently.

1.1 Scope of the Document

This document provides the Low-Level Design of the **Agentic AI Manufacturing System**. It includes:

- Internal architecture of multi-agent workflow
- Agent design and responsibilities
- Task orchestration mechanism
- Data and process flow
- API and LLM integration
- Design considerations

1.2 Intended Audience

This document is intended for the following stakeholders:

- Software Developers – Implementation reference
-

- System Architects – Design understanding
- Faculty / Evaluators – Project assessment
- Students – Learning agentic AI systems

1.3 System Overview

The Agentic AI system is designed to process user queries related to manufacturing workflows and generate structured, decision-ready outputs using collaborative AI agents.

Unlike traditional AI systems that rely on a single prompt-response mechanism, this system introduces **task decomposition and agent specialization**. The user query is broken down into smaller subtasks, which are then executed sequentially by different agents.

The workflow of the system includes:

1. **Information Retrieval** – Conducted by the Researcher Agent
2. **Data Structuring (JSON)** – Ensures machine-readable output
3. **Analysis & Recommendation** – Performed by the Writer Agent
4. **Final Presentation** – Displayed via Streamlit UI

The system is intentionally **domain-restricted to manufacturing workflows**, which improves response accuracy, reduces ambiguity, and ensures that generated outputs are relevant and reliable.

2. System Design

2.1 Application Design

The system follows a **three-tier architecture** consisting of:

The system follows a **layered architectural pattern**, ensuring separation of concerns and maintainability.

Presentation Layer

The presentation layer is implemented using **Streamlit**, which provides an interactive and lightweight interface for user interaction. It allows users to input queries and view generated outputs in real time.

This layer is responsible only for interaction and does not contain any business logic, ensuring a clean separation between UI and backend processing.

Application Layer

The application layer serves as the core processing unit of the system. It is implemented in Python and handles:

- Agent creation and configuration
- Task definition and assignment
- Workflow orchestration
- Data flow between components

The run() function acts as the central controller, coordinating all system operations from input handling to final output generation.

Agentic AI Layer

This layer represents the intelligence of the system. It consists of multiple AI agents, each with:

- A defined role
- A specific goal
- Contextual backstory

This design enables agents to behave like domain experts, improving the relevance and quality of outputs.

The use of **LLMs as cognitive engines** allows agents to:

- Understand natural language
- Generate structured outputs
- Perform reasoning and comparison

2.2 Process Flow

Workflow begins when the user submits a query through the frontend interface. This query is passed to the backend, where the main orchestration logic is executed.

The system dynamically creates agents and assigns them tasks. The Researcher Agent executes the first task, generating structured data in JSON format. This output is then passed as context to the Writer Agent, which performs analysis and generates a final report.

This step-by-step execution ensures:

- Logical flow of information
- Reduced errors
- Improved traceability of outputs

2.3 Information Flow

User Input

↓

Streamlit UI (app.py)

↓

run() Function

↓

main.py (Agent Orchestration)

↓

Researcher Agent

↓

JSON Output

↓

Writer Agent

↓

Final Report

↓

Frontend Display

2.4 Components Design

1. User Interface Module (app.py)

- Accepts user input
- Calls backend
- Displays output

2. Configuration Module (config.py)

- Loads API keys
- Configures LLM

3. Agent Module

Researcher Agent

- Role: Supplier Research Specialist
- Output: Structured JSON

Writer Agent

- Role: Report Generator
- Output: Final formatted report

4. Task Module

Research Task

- Finds supplier details
- Outputs strict JSON

Write Task

- Uses JSON
- Generates insights & recommendations

5. Orchestration Module (main.py)

- Creates agents
- Assigns tasks
- Manages execution using CrewAI

- Passes context between tasks

2.5 Key Design Considerations

Several factors were considered while designing the system:

Scalability

The system should handle multiple user requests and support future integration of additional AI models.

Performance

Efficient API calls and response handling ensure faster output generation.

Usability

The interface should be simple and easy for users without technical knowledge.

Reliability

Proper error handling ensures the system functions even when APIs fail or network issues occur.

Security

API keys and credentials must be securely stored to prevent unauthorized access.

2.6 API Catalogue

1. Domain Restriction

System only accepts manufacturing-related queries to improve accuracy.

2. Modularity

Each agent has a single responsibility → improves maintainability.

3. Scalability

New agents (e.g., Pricing Agent, Logistics Agent) can be added easily.

4. Structured Communication (JSON)

Ensures:

- Clean data transfer
- No ambiguity
- Better reliability

5. Performance

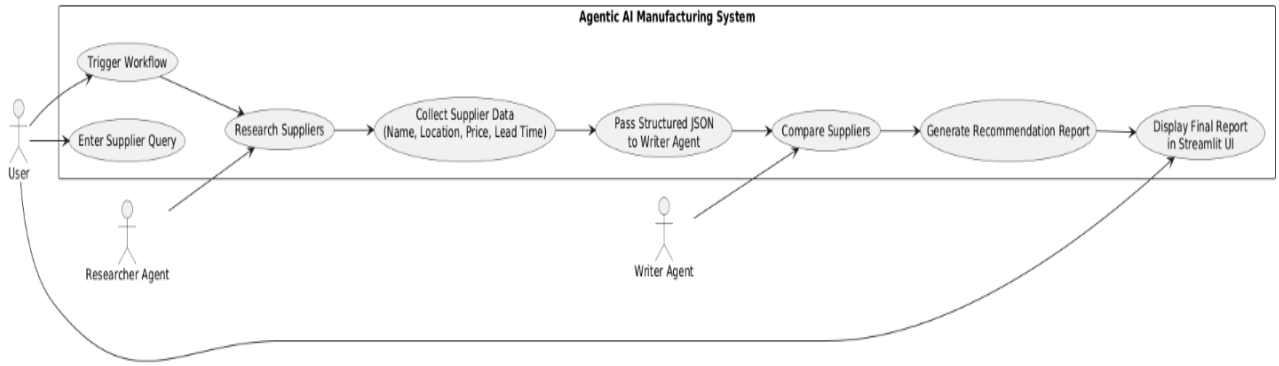
Sequential task execution ensures controlled workflow.

6. Reliability

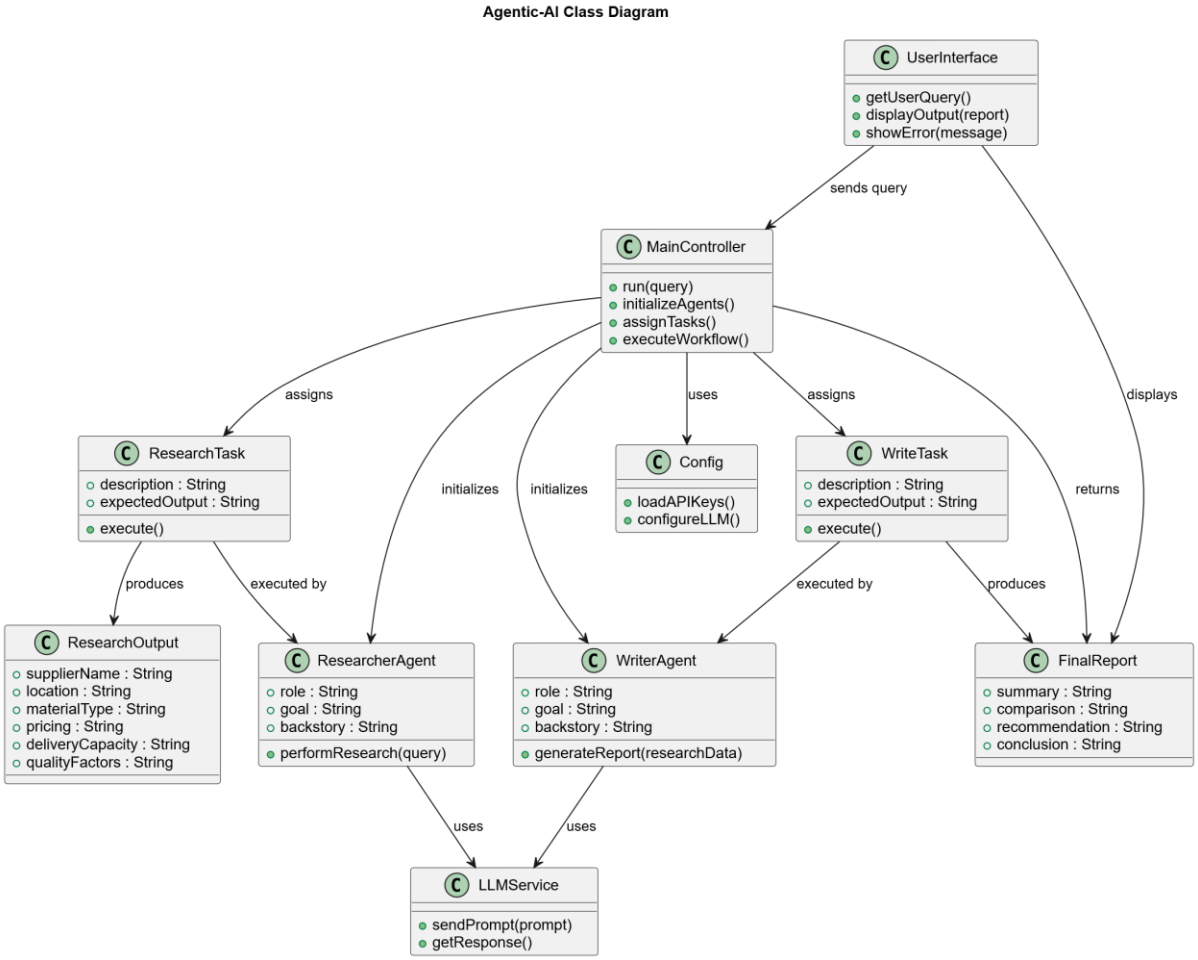
- Error handling in API calls
- Validation of user input

4.Architecture Diagram

Use Case Diagram

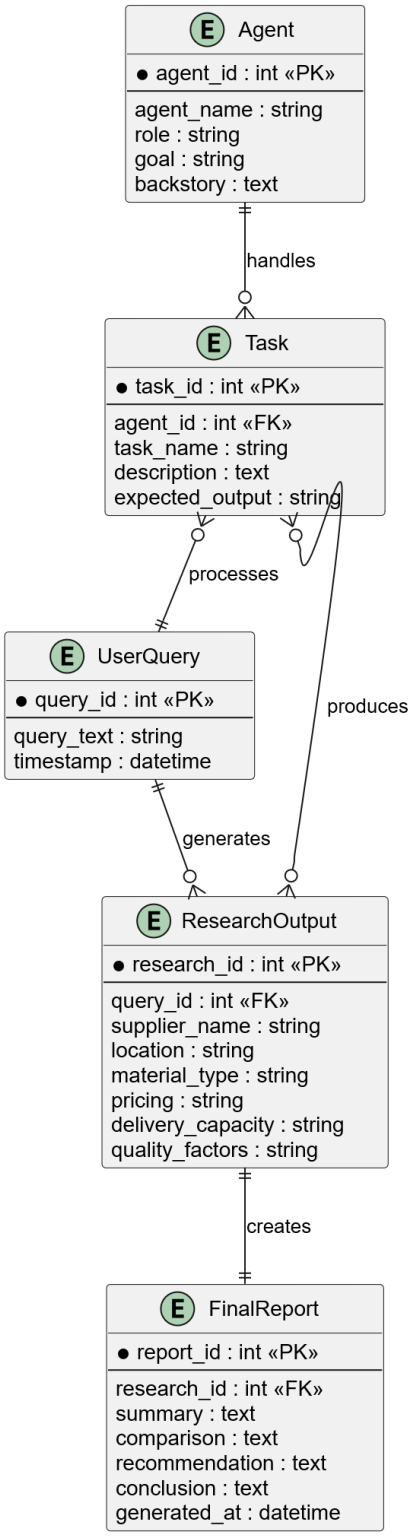


CLASS Diagram

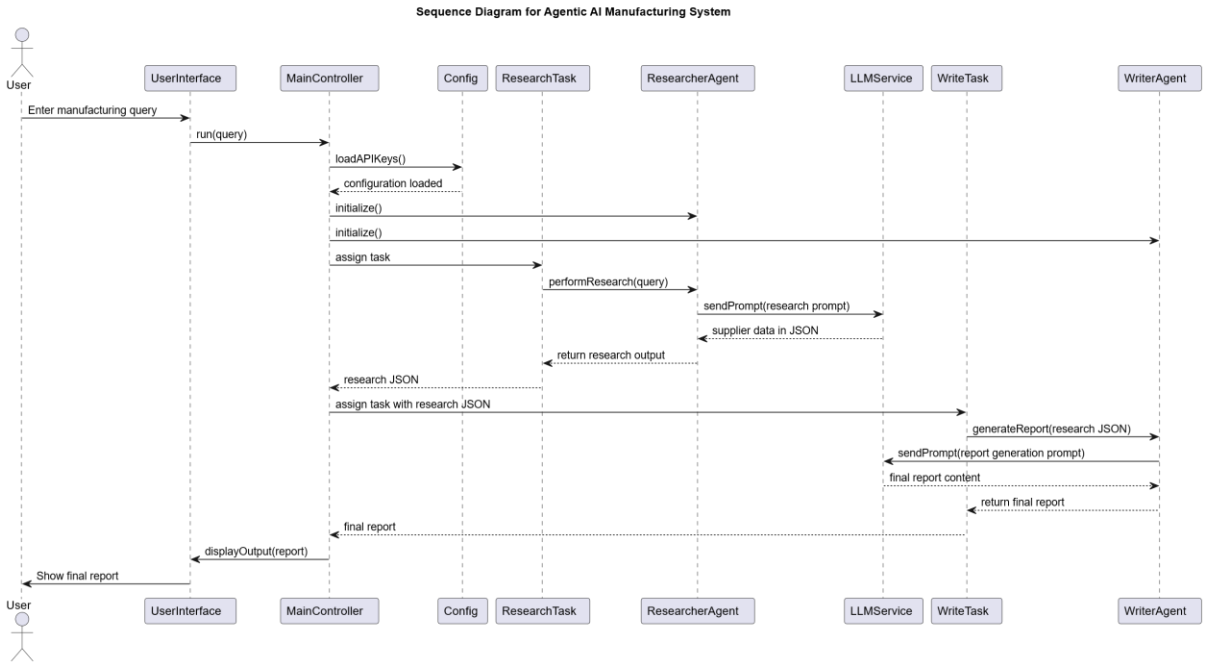

Agentic-AI Class Diagram
UserInterface
getUserQuery()
displayOutput(report)
showError(message)
sends query
displays
MainController
run(query)
initializeAgents()
assignTasks()
executeWorkflow()
assigns
uses
assigns
initializes
initializes
returns
ResearchTask
description : String
expectedOutput : String
execute()
Config
loadAPIKeys()
configureLLM()
WriteTask
description : String
expectedOutput : String
execute()
produces
executed by
executed by
produces
ResearchOutput
supplierName : String
location : String
materialType : String
pricing : String
deliveryCapacity : String
qualityFactors : String
ResearcherAgent
role : String
goal : String
backstory : String
performResearch(query)
WriterAgent
role : String
goal : String
backstory : String
generateReport(researchData)
FinalReport
summary : String
comparison : String
recommendation : String
conclusion : String
uses
uses
LLMService
sendPrompt(prompt)
getResponse()

ER Diagram

Agentic AI ER Diagram



SEQUENCE Diagram



Activity Diagram

3. Data Design

3.1 Data Model

The Agentic AI Manufacturing System uses a **structured, task-oriented data model** in which information flows between modules and agents in a controlled manner. Unlike traditional enterprise applications that rely heavily on relational tables and persistent records, this system primarily processes **runtime data objects** generated from user input, agent responses, and intermediate task outputs. The central design principle is that each stage of execution produces a clearly defined output which becomes the input for the next stage.

The first important data entity is the **User Query Object**. This object represents the manufacturing-related input entered by the user through the Streamlit interface. It may contain the problem statement, supplier search requirement, material requirement, or decision-making request. This query acts as the trigger for the complete workflow.

The second entity is the **Research Output Object**. This is generated by the Researcher Agent after performing the supplier analysis task. The output is expected in **JSON format**, because JSON provides a machine-readable and structured representation of the collected information. This object may include fields such as supplier name, product/material category, location, pricing indicator, quality factors, delivery capacity, and other business-relevant parameters. By enforcing a structured JSON schema, the system reduces ambiguity and ensures consistency across agent-to-agent communication.

The third entity is the **Writer Input Context Object**, which is the processed research output passed as contextual input to the Writer Agent. This object does not merely transfer raw information; it acts as a refined knowledge package that allows the Writer Agent to generate meaningful analysis, comparisons, and recommendations.

The fourth entity is the **Final Report Object**. This contains the final user-facing report generated by the Writer Agent. The report is usually presented in readable textual form and includes supplier comparison, analysis, recommendations, and structured decision support output. Although displayed as text in the UI, it is conceptually treated as a separate output object because it represents the final business value delivered by the system.

The fifth entity is the **Agent Configuration Object**. Each agent in the system has attributes such as role, goal, backstory, expected behavior, and assigned task. This configuration helps the LLM behave like a specialized domain expert rather than a general-purpose chatbot. In design terms, this object defines the identity and capability of each agent.

Thus, the system's data model is not database-table-centric, but **workflow-centric**, where data is dynamically created, transformed, validated, passed forward, and finally presented to the user. This model is well suited for intelligent multi-agent systems where the value lies in orchestration and reasoning rather than permanent transactional storage.

3.2 Data Access Mechanism

The current system uses an **in-memory data access mechanism**, which means that data is processed during execution and transferred between components without relying on a persistent database. This design is suitable for lightweight academic prototypes and proof-of-concept agentic systems, because it reduces implementation complexity and keeps the architecture modular.

The data access process begins at the **presentation layer**, where the user enters a query in the Streamlit interface. Once the query is submitted, it is passed to the application logic, where the run() function or main orchestration module initiates workflow execution. This module acts as the central controller and manages which agent receives which task.

The Researcher Agent receives the original query and generates a structured output. This output is then stored temporarily in memory as a Python object or a JSON response. The Writer Agent accesses this temporary object as input context and uses it to generate the final analytical report. Therefore, data access in this system is sequential and context-driven rather than query-driven.

The major mechanisms of data access in this architecture are:

1. Function-Level Data Transfer

Data moves between modules through Python function arguments and return values. This is the simplest and most direct way to maintain flow between UI, orchestrator, tasks, and agents.

2. Context-Based Agent Communication

CrewAI-style orchestration allows one task's output to be used as context for the next task. This is a major aspect of the system because it enables inter-agent collaboration without tightly coupling their internal logic.

3. Temporary Runtime Objects

Intermediate outputs exist only during the active execution cycle. They are not committed to long-term storage unless explicitly added in future versions.

4. Configuration-Based Access

API keys, model configuration, and agent settings are loaded through environment variables and configuration files. This ensures secure and centralized access to operational parameters.

At present, the system does not use SQL queries, ORMs, or direct database reads/writes. However, the architecture is flexible enough to support future enhancements. If persistent data storage becomes necessary, the following extensions can be added:

- MongoDB for storing reports, sessions, and supplier records
- PostgreSQL/MySQL for structured enterprise reporting
- Vector databases for semantic memory and retrieval
- Cloud storage for report archival

This design keeps the current implementation lightweight while leaving room for scalable enterprise adoption in the future.

3.3 Data Retention Policies

The system is designed with a **minimal retention approach**, meaning that data is primarily used for immediate execution and is not permanently stored in the current implementation. This policy aligns with privacy, simplicity, and academic project requirements.

In the present version, user queries, research outputs, and generated reports remain available only during the active execution cycle or application session. Once the session ends or the application is refreshed, the temporary data is lost unless explicitly saved. This behavior is

beneficial because it reduces storage complexity and avoids unnecessary retention of potentially sensitive business information.

The data retention policy of the system can be understood in three layers:

1. Runtime Retention

All intermediate objects such as user query, JSON supplier data, and generated report are retained only in memory while the task is active.

2. Session-Level Retention

If Streamlit session state or temporary variables are used, outputs may remain available during the current interactive session but are not guaranteed beyond that.

3. No Permanent Storage by Default

The current architecture does not maintain a database of past queries, supplier research results, or generated manufacturing decisions.

This retention strategy has several advantages:

- Improves user privacy
- Reduces security risk from stored sensitive data
- Minimizes storage overhead
- Simplifies architecture and maintenance

However, for enterprise-level adoption, more formal retention policies may be required. In such cases, the following policies can be introduced:

- Retain reports for a fixed duration such as 30, 60, or 90 days
- Archive important supplier analysis results for historical comparison
- Automatically delete outdated records after the retention period
- Maintain audit logs for system usage and report generation
- Store only anonymized or consent-based data

Thus, the current policy is intentionally lightweight and privacy-friendly, while future versions can adopt configurable retention based on organizational needs.

3.4 Data Migration

At present, the project does not require conventional data migration because the core system does not depend on a persistent database. Since the workflow is mainly runtime-driven and outputs are temporary, there are no schema evolution challenges or database version transitions in the current implementation.

However, from a design perspective, it is important to define how migration would be handled if the system evolves into a production-ready platform. Future migration requirements may emerge when the application starts storing supplier data, report history, user profiles, or analytical results.

Possible migration scenarios include:

1. JSON to Database Migration

Currently, supplier research results are generated as JSON objects. In the future, these JSON records can be migrated into document-based databases such as MongoDB. This would allow historical storage and searchability.

2. Session Data to Persistent Report Storage

Generated reports that are currently displayed only in the UI may later be stored in structured collections or relational tables for audit and review.

3. Local to Cloud Migration

If the system is deployed at scale, local file-based or temporary runtime storage may be migrated to cloud platforms such as AWS, Azure, or Google Cloud.

4. Schema Evolution

If new agents are added—for example Pricing Agent, Logistics Agent, Risk Agent, or Inventory Agent—the data schema will expand. Migration scripts may then be needed to adjust old stored records to new formats.

5. Export and Import Support

The system may later support exporting analysis in CSV, Excel, or PDF format. Migrating from one reporting structure to another may require transformation logic.

To support future migration, the following design principles are recommended:

- Keep JSON schemas consistent and documented
- Use versioned data formats where necessary
- Separate business logic from storage logic
- Provide data transformation utilities
- Validate migrated records before production use

Therefore, although migration is not a current runtime necessity, defining migration readiness in HLD demonstrates that the system is designed with future scalability and maintainability in mind.

4. Interfaces

The Interfaces section defines how different parts of the system communicate with one another and how the system interacts with users and external services. In the Agentic AI Manufacturing System, interfaces play an essential role because the entire workflow depends on **clear, structured communication** between UI, orchestration logic, agents, and LLM services.

4.1 User Interface

The user interface is built using **Streamlit**, which provides a simple and interactive web-based frontend. The interface allows the user to enter manufacturing-related queries and view the generated report in an easy-to-read format. The UI is intentionally minimal so that non-technical users can operate the system without any training.

The primary responsibilities of the user interface are:

- Accept user input

- Trigger the backend processing flow
- Show loading or execution progress
- Display final output in a readable form
- Present errors or invalid input messages when necessary

This interface does not contain heavy business logic, which helps maintain separation of concerns.

4.2 Application Interface

The application interface refers to the internal communication between the frontend and backend execution modules. In this system, the UI invokes the backend through direct Python function calls rather than REST endpoints. The run() or main orchestration function acts as the control interface of the application.

This interface is responsible for:

- Receiving user queries from the frontend
- Initializing agents
- Assigning tasks
- Managing execution order
- Returning final output to the UI

Because the application is implemented in Python as a single logical stack, this interface is lightweight, fast, and easy to maintain.

4.3 Agent-to-Agent Interface

A defining feature of this project is the communication between multiple specialized AI agents. The Researcher Agent and Writer Agent do not communicate in an unstructured conversational way; instead, they exchange information through **task outputs and structured context**.

The Researcher Agent produces supplier-related findings in JSON format. This structured output forms the interface contract for the Writer Agent. The Writer Agent consumes that structured information and converts it into meaningful analysis and recommendations.

The agent-to-agent interface ensures:

- Controlled data transfer
- Reduced ambiguity
- Better reliability in downstream tasks
- Easier debugging of intermediate outputs

This is one of the most important interface designs in the project because it enables modular and scalable multi-agent collaboration.

4.4 External API / LLM Interface

The intelligence of the system is powered through LLM integration. The application communicates with language model services using secure API-based access. The model API acts as the cognitive engine behind both the Researcher Agent and the Writer Agent.

This interface is responsible for:

- Sending prompts to the model
- Receiving generated responses
- Handling structured output requirements
- Managing token-based interaction with the LLM

The API key is stored securely using environment variables, and the application accesses it through configuration logic. This avoids hardcoding sensitive credentials into source code.

4.5 Input and Output Interface

The input interface accepts manufacturing-related natural language queries. The output interface returns either a structured supplier analysis or a final written report. The output is displayed directly in the Streamlit frontend, but in future versions it may be exported to files such as PDF, DOCX, CSV, or Excel.

Overall, the interface design of the system is modular, clean, and aligned with the layered architecture described earlier.

5. State and Session Management

State and session management describe how the system handles temporary data, user execution context, and intermediate outputs during the workflow lifecycle. Since the current project is a lightweight interactive application, it does not require enterprise-grade session control, but it still relies on controlled runtime state handling.

When the user submits a query, the system enters an active processing state. During this period, several intermediate objects exist simultaneously: the original user query, the Researcher Agent's JSON output, the Writer Agent's response, and any execution messages or errors. These must remain accessible while the workflow is running.

The system currently uses **request-level and session-level state management**, which can be understood as follows:

5.1 Request Lifecycle

Each submitted query creates a new execution cycle. This cycle begins when the query is entered and ends when the final output is displayed. During this lifecycle, all temporary data belongs only to that request.

The typical sequence is:

- User enters query
- Backend receives query
- Agents are initialized
- Research task executes
- JSON context is created
- Write task executes
- Final output is returned

- Temporary runtime state is discarded or reset

This model is simple and effective for single-user and low-complexity scenarios.

5.2 Agent Context Handling

The system uses agent context as a temporary state carrier. The Writer Agent depends on the output of the Researcher Agent, so that structured output must remain available during workflow execution. This is not stored permanently, but it is preserved in memory until all tasks complete.

This context-based design ensures that agents remain logically independent while still being able to collaborate in sequence.

5.3 Streamlit Session State

If Streamlit session state is used, the application can retain limited information during user interaction, such as:

- Last submitted query
- Latest generated output
- Status messages
- Processing flags

This can improve user experience by preventing accidental loss of data during a single interactive session.

5.4 Session Limitations

The current implementation does not include:

- User authentication sessions
- Multi-user account isolation
- Long-term history tracking
- Persistent user dashboards

These limitations are acceptable for an academic prototype, but future versions can include:

- Login-based session control
- Role-based access
- User-specific report history
- Concurrent multi-user state isolation

Thus, the current state and session management strategy is lightweight, execution-oriented, and appropriate for the scope of the project.

6. Caching

Caching is the process of storing frequently used or previously generated results so that repeated requests can be served more quickly. In the current implementation, caching is limited or optional, but it is an important design consideration for future system optimization.

The present version of the application generates outputs dynamically for each query. This ensures freshness, but it may also lead to repeated processing cost if similar requests are submitted again. Therefore, even though caching is not heavily implemented now, it is valuable to define how it can be incorporated into the architecture.

6.1 Need for Caching

The system interacts with AI models and possibly external supplier information sources. These operations may consume time and API cost. If the same or similar prompt is repeatedly processed, caching can significantly reduce latency and improve efficiency.

Caching is useful for:

- Faster response generation
- Lower API usage cost
- Reduced repeated reasoning load
- Better user experience for repeated queries

6.2 Cacheable Components

Several components of the system can benefit from caching:

1. Repeated User Query Results

If the same manufacturing query is entered multiple times, the final report can be reused instead of regenerated from scratch.

2. Researcher Agent Output

Supplier analysis results in JSON format can be temporarily cached so that the Writer Agent does not need to wait for repeated upstream research.

3. Configuration and Initialization Objects

Model settings, agent configuration, and task setup can also be cached to reduce repeated initialization overhead.

4. Session-Level Outputs

Within one active user session, intermediate outputs can be retained so the UI can re-display results without rerunning the full workflow.

6.3 Caching Strategy

A suitable caching strategy for this project may include:

- In-memory caching for short-term repeated queries
- Session caching for interactive use
- Time-based expiration to ensure data freshness
- Hash-based key generation using query text

For advanced versions, tools such as Redis or Python caching libraries may be integrated.

6.4 Cache Invalidation

Cache invalidation is important to prevent stale or outdated results. In this project, cached results should be cleared when:

- The session ends
- The supplier data becomes outdated
- The underlying task prompt changes
- The model or logic is updated

Thus, caching should improve speed without compromising correctness and relevance.

7. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system beyond its functional behavior. In this project, they ensure that the Agentic AI system is not only correct in terms of workflow but also secure, efficient, scalable, reliable, and easy to maintain.

7.1 Security Aspects

Security is a major consideration because the system depends on external model APIs and may process manufacturing-related business queries. The first requirement is **secure API key management**. Credentials must not be hardcoded in the source code; instead, they should be stored in environment files or secure deployment configurations.

The second requirement is **input validation**. Since the system accepts natural language input, it must validate empty queries, unsupported domain requests, or malformed instructions. This prevents misuse and improves output quality.

The third requirement is **controlled configuration access**. Only authorized developers or maintainers should have access to sensitive files containing model keys or deployment configuration.

The fourth requirement is **safe logging**. Logs should record useful system events without exposing sensitive user prompts, private supplier details, or secret keys.

The fifth requirement is **secure deployment communication**. If deployed online, the application should use HTTPS and secure hosting practices to protect data in transit.

Future security enhancements may include:

- User authentication and authorization
- Role-based access control
- Encrypted report storage
- Audit trails
- Rate limiting and abuse detection

7.2 Performance Aspects

Performance defines how quickly and efficiently the system responds to user input. Since the project uses multiple AI agents, response time depends on prompt complexity, API latency, and sequential task execution.

The current design improves performance by keeping the architecture lightweight and modular. The Streamlit frontend is simple, and data transfer between layers happens in memory. This reduces unnecessary I/O overhead.

Important performance factors include:

- Efficient orchestration of tasks
- Clear prompts to minimize token waste
- Structured JSON output to simplify downstream processing
- Limited number of agent transitions
- Controlled sequential execution

Performance goals for the system include:

- Fast acceptance of user input

- Reasonably quick generation of supplier research
- Smooth handoff from research task to writing task
- Minimal delay in final report display

Future performance improvements may include:

- Query/result caching
- Async execution where appropriate
- Optimized model selection
- Load balancing for multiple users
- External memory/vector store integration

7.3 Scalability

Scalability refers to the system's ability to grow without major redesign. The multi-agent architecture already supports scalability because new agents can be added as separate modules. For example, future versions may include a Pricing Agent, Logistics Agent, Quality Agent, or Risk Analysis Agent.

Scalability can also be extended in the following ways:

- Support for multiple concurrent users
- Integration with persistent databases
- Cloud deployment for broader access
- Domain expansion beyond manufacturing

Thus, the modular architecture provides a strong foundation for future growth.

7.4 Reliability

Reliability means the system should produce dependable results even when facing temporary errors or incomplete inputs. Since the system depends on external APIs, it must be able to handle failures gracefully.

Reliability considerations include:

- Error handling around LLM/API calls
- Fallback messages when a task fails
- Validation of intermediate JSON outputs
- Clear reporting of exceptions to the UI
- Avoiding silent failures

A reliable system should continue to behave predictably even if one component temporarily underperforms.

7.5 Maintainability

Maintainability is one of the strongest design features of this project. The architecture separates concerns into multiple modules such as UI, configuration, agents, tasks, tools, memory, and orchestration. This makes the code easier to read, debug, and extend.

Maintainability is improved by:

- Modular file structure
- Single responsibility for each agent
- Structured tasks
- Reusable configuration logic
- Clear separation between interface and execution logic

Because of this, future developers can add features or fix bugs without affecting the entire application.

7.6 Usability

The system is designed to be usable even by users with limited technical background. The Streamlit interface is simple, direct, and focused only on the essential user journey: enter query, run workflow, view result.

Usability characteristics include:

- Minimal learning curve
- Simple interaction model
- Human-readable output
- Clean and direct interface
- Reduced clutter and complexity

This makes the project suitable not only as a technical prototype but also as a teaching and demonstration tool.

1. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press.
2. Official documentation of generative AI APIs and LLM integration frameworks.
3. Official documentation of multi-agent orchestration frameworks used for task-based agent collaboration.
4. Streamlit official documentation for Python-based UI development.
5. Research papers on Agentic AI, Multi-Agent Systems, and structured reasoning workflows.
6. Publications related to Generative AI applications in manufacturing and decision support systems.
7. Python official documentation for backend and configuration handling.

This reference section supports both the theoretical and practical foundations of the project and shows that the system is based on established AI concepts as well as modern implementation tools.

8. References

1. Goodfellow, I., Bengio, Y., & Courville, A. – *Deep Learning*
2. OpenAI Documentation – Generative AI APIs
3. Hugging Face Transformers Documentation
4. Research papers on **Multimodal Artificial Intelligence**
5. Generative AI Applications in Manufacturing – IEEE Publications